



中国海洋大学
OCEAN UNIVERSITY OF CHINA

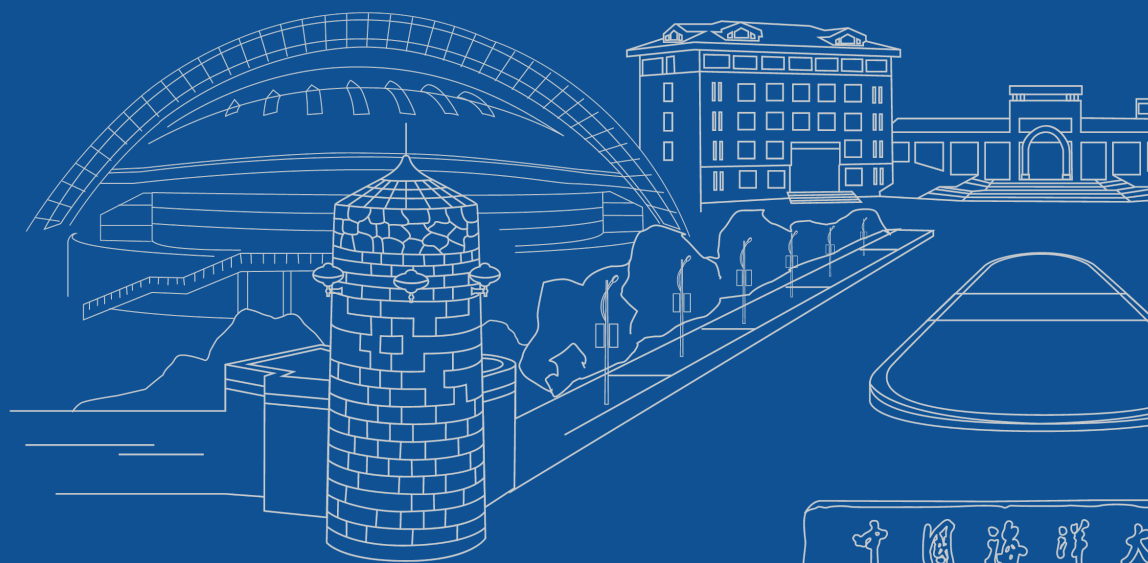
第二周实验报告

系统开发工具基础

刘晨希

25020007076

信息科学与工程学部
计算机科学与技术学院
计算机科学与技术专业
系统开发工具基础



中国海洋大学

2026 年 9 月 10 日



第二周实验报告

系统开发工具基础

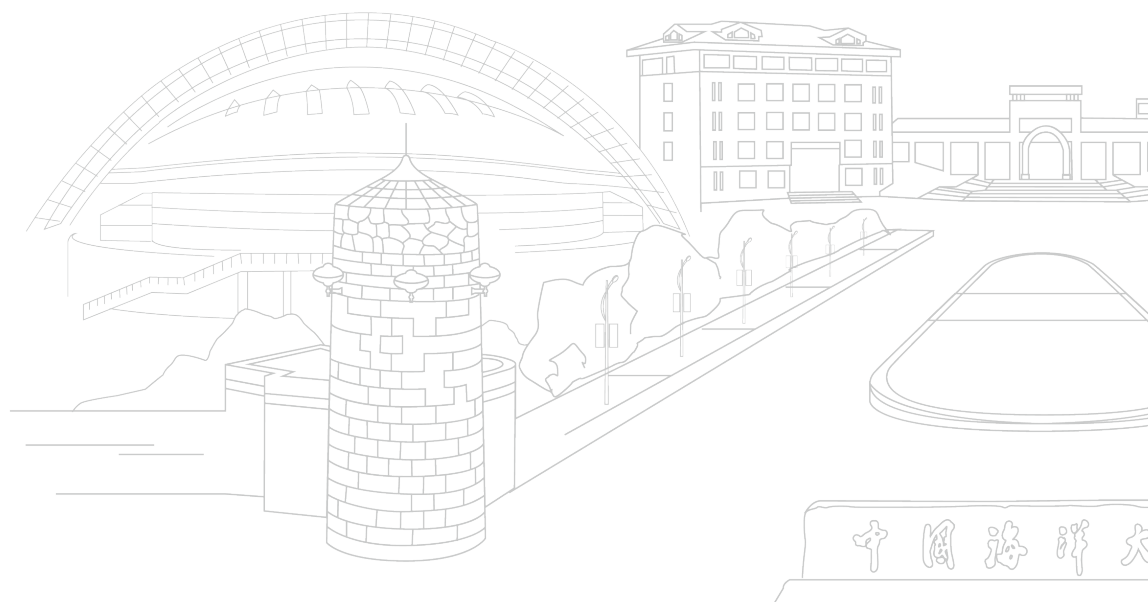
刘晨希

25020007076

指导老师: 周小伟

信息科学与工程学部
计算机科学与技术学院
计算机科学与技术专业
系统开发工具基础

2026 年 9 月 10 日



目录

目录	i
1 课堂练习及结果	2
1.1 第 5 题控制一个可清理的后台任务	2
1.1.1 实验内容	2
1.1.2 实验结果	3
1.2 第 6 题语义重构与本地开发反馈	4
1.2.1 实验内容	4
1.2.2 实验结果	7
1.3 第 7 题用调试器定位归并排序缺陷	7
1.3.1 实验内容	7
1.3.2 实验结果	9
1.4 第 8 题先测量，再优化慢速词频程序	9
1.4.1 实验内容	9
1.4.2 实验结果	11
1.5 Github 仓库地址	11
2 课后练习内容和结果	12
2.1 第 1 题花括号展开	12
2.1.1 实验内容	12
2.2 第 2 题重定向	12
2.2.1 实验内容	12
2.3 第 3 题引号	13
2.3.1 实验内容	13
2.4 第 4 题退出码	13
2.4.1 实验内容	13
2.5 第 5 题作业控制	14
2.5.1 实验内容	14
2.6 第 6 题别名	15
2.6.1 实验内容	15
2.7 第 7 题 Vim 插入模式	15
2.7.1 实验内容	15
2.8 第 8 题 Vim 模态编辑	16
2.8.1 实验内容	16
2.9 第 9 题 time 计时命令	16

目录	1
2.9.1 实验内容	16
2.10 第 10 题 pdb 调试器	17
2.10.1 实验内容	17
3 解题感悟	19

课堂练习及结果

1.1 第 5 题控制一个可清理的后台任务

1.1.1 实验内容

如图 1.1 所示,使用 vim 编辑器创建 worker.sh。脚本通过 trap 命令设置了 TERM 和 INT 两个信号的处理函数,当脚本收到这两个信号时,会执行 echo CLEAN_EXIT >> cleanup.log; exit 0,即向 cleanup.log 写入 CLEAN_EXIT 后以退出码 0 正常结束;随后进入无限循环,每秒钟打印一次计数器 n 并将其自增。

```
liuchenxi@liuchenxideMacBook-Air q05 % vim worker.sh
```

图 1.1: 创建 worker.sh

如图 1.2 所示,使用 chmod +x 命令为 worker.sh 添加执行权限。

```
liuchenxi@liuchenxideMacBook-Air q05 % chmod +x worker.sh
```

图 1.2: 为 worker.sh 添加执行权限

如图 1.3 所示,在前台启动 worker.sh,并通过重定向把 stdout 写入 stdout.log、stderr 写入 stderr.log;随后使用 Ctrl-Z 挂起该任务,从图中可以看到 zsh 输出了 suspended 提示,说明任务已被挂起。

```
liuchenxi@liuchenxideMacBook-Air q05 % ./worker.sh > stdout.log 2> stderr.log  
^Z  
zsh: suspended ./worker.sh > stdout.log 2> stderr.log
```

图 1.3: 前台启动 worker.sh 并挂起

如图 1.4 所示,使用 bg 命令让被挂起的任务转到后台继续运行,并使用 jobs 命令确认任务状态。从图中可以看到,jobs 输出 [1] + running ./worker.sh > stdout.log 2> stderr.log,说明任务正在后台运行。

```
liuchenxi@liuchenxideMacBook-Air q05 % bg  
[1] + continued ./worker.sh > stdout.log 2> stderr.log  
liuchenxi@liuchenxideMacBook-Air q05 % jobs  
[1] + running ./worker.sh > stdout.log 2> stderr.log
```

图 1.4: 使用 bg 让任务后台继续运行

如图 1.5 所示, 使用 `jobs -p` 取得后台任务的进程号 (PID), 并通过 `kill` 命令发送 `SIGTERM` 信号结束任务, 全程没有手工抄写 PID, 也没有使用 `kill -9`。任务收到 `SIGTERM` 信号后触发了 `trap` 中设置的清理逻辑, 最终以 `[1] + done` 结束 (`done` 表示进程以退出码 0 正常退出, 而非被信号强制终止), 随后 `jobs` 命令输出为空, 说明任务已经退出。

```
liuchenxi@liuchenxideMacBook-Air q05 % kill -TERM $(jobs -p)
kill: illegal pid: [1]
kill: illegal pid: +
kill: illegal pid: running
kill: illegal pid: ./worker.sh
kill: illegal pid: >
kill: illegal pid: stdout.log
kill: illegal pid: 2>
kill: illegal pid: stderr.log
liuchenxi@liuchenxideMacBook-Air q05 %
[1] + done      ./worker.sh > stdout.log 2> stderr.log
liuchenxi@liuchenxideMacBook-Air q05 % jobs
```

图 1.5: 发送 `SIGTERM` 信号结束任务

在图 1.5 中, `kill` 为发送信号命令, `-TERM` 表示发送 `SIGTERM` (终止) 信号, `jobs -p` 用于输出后台任务的进程号。需要注意的是, `zsh` 中直接执行 `kill -TERM $(jobs -p)` 时, 命令替换 `$(jobs -p)` 展开后得到的是任务的完整状态行而非纯 PID, 导致出现 `illegal pid` 报错, 需要先正确提取出纯 PID 再发送信号。

如图 1.6 所示, 使用 `grep` 命令在 `cleanup.log` 中查找 `CLEAN_EXIT`, 输出结果为 `CLEAN_EXIT`, 说明脚本的 `trap` 清理逻辑被正确触发, 任务已经干净地退出。

```
liuchenxi@liuchenxideMacBook-Air q05 % grep "CLEAN_EXIT" cleanup.log
CLEAN_EXIT
```

图 1.6: 确认 `cleanup.log` 中出现 `CLEAN_EXIT`

1.1.2 实验结果

第五题的 github 的 commit 记录和 push 记录如图 1.7 所示。

```
liuchenxi@liuchenxideMacBook-Air w2 % git add q05
liuchenxi@liuchenxideMacBook-Air w2 % git commit -m "完成了q05"
[main 9f36f10] 完成了q05
 4 files changed, 110 insertions(+)
 create mode 100644 w2/q05/cleanup.log
 create mode 100644 w2/q05/stderr.log
 create mode 100644 w2/q05/stdout.log
 create mode 100755 w2/q05/worker.sh
liuchenxi@liuchenxideMacBook-Air w2 % git push
[Enter passphrase for key '/Users/liuchenxi/.ssh/id_ed25519':
Enumerating objects: 9, done.
Counting objects: 100% (9/9), done.
Delta compression using up to 10 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (8/8), 764 bytes | 764.00 KiB/s, done.
Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:lcx821/lcx_course_work.git
   b92e9c1..9f36f10  main -> main
```

图 1.7: github 记录 Q5

1.2 第 6 题语义重构与本地开发反馈

1.2.1 实验内容

按照题目给定内容,在 q06 目录中创建 `math_utils.py`、`app.py` 和 `test_math_utils.py` 三个 Python 文件,并在 VSCode 中打开 q06,启用 Pylance (Python 语言服务器)。其中 `math_utils.py` 定义了函数 `total_price`, `app.py` 和 `test_math_utils.py` 分别导入并调用、测试该函数。

如图 1.8 和图 1.9 所示,在 `app.py` 中把光标放到 `total_price` 上,Pylance 悬浮显示该函数的定义信息(参数类型和返回类型);再通过“跳转到定义”(Go to Definition)定位到 `math_utils.py` 中函数 `total_price` 的定义处。

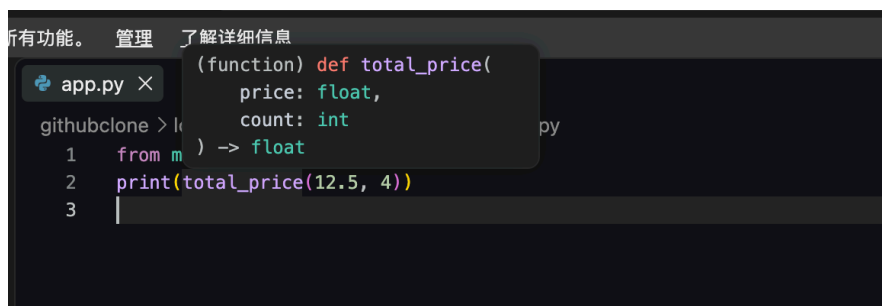


图 1.8: 悬浮显示 `total_price` 定义

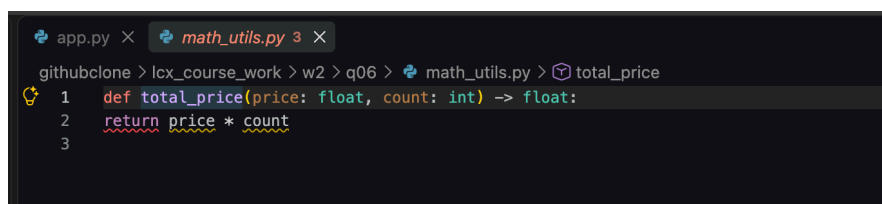


图 1.9: 跳转到 `math_utils.py` 中的定义

如图 1.10 所示,使用“查找所有引用”(Find All References)功能,可以看到 `total_price` 在 `app.py`、`math_utils.py` 和 `test_math_utils.py` 中的全部引用位置。

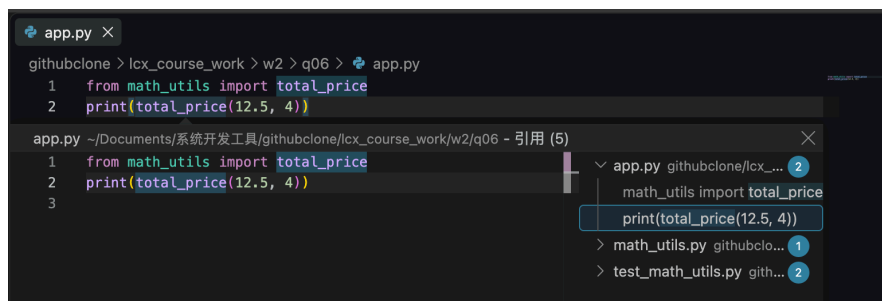


图 1.10: 查找 `total_price` 的所有引用

如图 1.11 和图 1.12 所示,使用“重命名符号”(Rename Symbol)把 `total_price` 重命名为 `calculate_total`,语言服务器会同步修改函数定义以及所有代码引用,保证定义与引用一致。

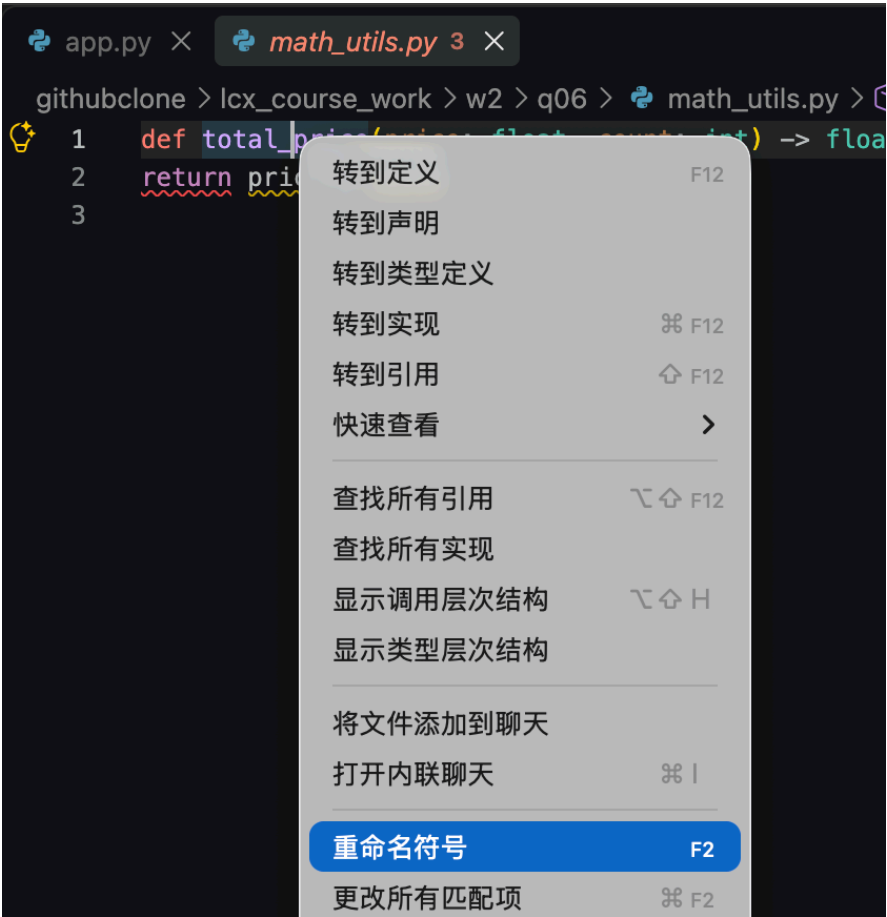


图 1.11: 使用重命名符号重命名 `total_price`

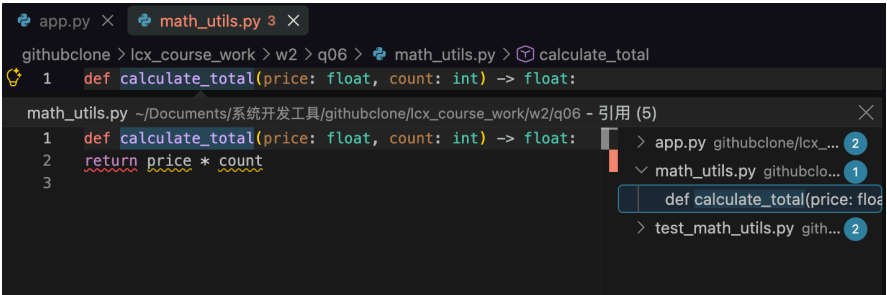


图 1.12: 重命名完成后的 `calculate_total`

如图 1.13 所示，在 `app.py` 中临时加入未使用的 `import os`，用于后续使用 `ruff` 检测并删除。

```

app.py  math_utils.py 3 x
githubclone > lcx_course_work > w2 > q06 > app.py
1  import os
2
3  from math_utils import calculate_total
4  print(calculate_total(12.5, 4))
5

```

图 1.13: 临时加入未使用的 `import os`

如图 1.14 所示，运行 `ruff check app.py`，`ruff` 检查出两个问题：F401 表示 `os` 被导入但未使用，I001 表示 `import` 块未排序或未格式化。

```

liuchenxi@liuchenxideMacBook-Air q06 % ruff check app.py
I001 [*] Import block is un-sorted or un-formatted
--> app.py:1:1

1 | / import os
2 | |
3 | | from math_utils import calculate_total
4 | | print(calculate_total(12.5, 4))
  | ^
help: Organize imports
3 | from math_utils import calculate_total
4 | +
5 | print(calculate_total(12.5, 4))

F401 [*] `os` imported but unused
--> app.py:1:8

1 | import os
  | ^^
2 |
3 | from math_utils import calculate_total
  |
help: Remove unused import: `os`
- import os
1 |

Found 2 errors.
[*] 2 fixable with the `--fix` option.

```

图 1.14: `ruff check` 发现未使用 `import`

如图 1.15 所示，运行 `ruff check --fix app.py` 自动修复，输出显示 2 个错误全部被修复。

```

liuchenxi@liuchenxideMacBook-Air q06 % ruff check --fix app.py
Found 2 errors (2 fixed, 0 remaining).

```

图 1.15: `ruff check --fix` 修复问题

如图 1.16 所示，运行 `pytest`，输出显示 1 个测试通过，说明重命名后的代码仍然正确。

```

liuchenxi@liuchenxideMacBook-Air q06 % ruff check .
All checks passed!
liuchenxi@liuchenxideMacBook-Air q06 % pytest
===== test session starts =====
platform darwin -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0
rootdir: /Users/liuchenxi/Documents/系统开发工具/githubclone/lcx_course_work/w2/q06
collected 1 item

test_math_utils.py . [100%]

===== 1 passed in 0.00s =====

```

图 1.16: pytest 测试通过

1.2.2 实验结果

第六题的 github 的 commit 记录和 push 记录如图 1.17 所示。

```

liuchenxi@liuchenxideMacBook-Air w2 % git add q06
liuchenxi@liuchenxideMacBook-Air w2 % git commit -m "完成了q06"
[main f1e18bb] 完成了q06
5 files changed, 11 insertions(+)
create mode 100644 w2/q06/__pycache__/math_utils.cpython-314.pyc
create mode 100644 w2/q06/__pycache__/test_math_utils.cpython-314-pytest-9.1.1.pyc
create mode 100644 w2/q06/app.py
create mode 100644 w2/q06/math_utils.py
create mode 100644 w2/q06/test_math_utils.py
liuchenxi@liuchenxideMacBook-Air w2 % git push
Enter passphrase for key '/Users/liuchenxi/.ssh/id_ed25519':
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 10 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (10/10), 1.99 KiB | 1.99 MiB/s, done.
Total 10 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:lcx821/lcx_course_work.git
9f36f10..f1e18bb main -> main

```

图 1.17: github 记录 Q6

1.3 第 7 题用调试器定位归并排序缺陷

1.3.1 实验内容

如图 1.18 所示, 使用 vim 编辑器创建 merge_sort.py, 内容为题目给定的存在缺陷的归并排序代码, 并用 cat 命令查看文件内容。其中 merge 函数 else 分支中的 result.append(right[i]) 处存在缺陷 (正确应为 right[j])。

```

liuchenxi@liuchenxideMacBook-Air q07 % vim merge_sort.py
liuchenxi@liuchenxideMacBook-Air q07 % cat merge_sort.py
def merge(left, right):
    result = []
    i = j = 0
    while i < len(left) and j < len(right):
        if left[i] <= right[j]:
            result.append(left[i]); i += 1
        else:
            result.append(right[i]); j += 1 # 此处存在缺陷
    return result + left[i:] + right[j:]

def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a) // 2
    return merge(merge_sort(a[:m]), merge_sort(a[m:]))

print(merge_sort([3, 1, 4, 1, 5, 9, 2, 6]))

```

图 1.18: 创建 merge_sort.py

如图 1.19 所示, 直接运行 `merge_sort.py`, 可以看到输出结果错误 (或抛出异常), 确认代码存在缺陷。

```
liuchenxi@liuchenxideMacBook-Air q07 % python3 merge_sort.py
[1, 5, 4, 3, 4, 5, 6, 9]
```

图 1.19: 运行 `merge_sort.py` 结果错误

如图 1.20 所示, 使用 `python3 -m pdb merge_sort.py` 启动调试器: 先用 `l` 命令查看源码, 再在 `merge` 函数的第 8 行 (缺陷行) 使用 `b 8` 设置断点, 用 `c` 命令继续执行到断点处, 然后使用 `p` 命令打印 `i`、`j`、`left[i]`、`right[j]` 的值。从图中可以看到, 此时 `i=0`、`j=0`、`left[i]=3`、`right[j]=1`, 即 `left[i] > right[j]`, 本应把较小的 `right[j]` 加入结果, 但缺陷代码取的是 `right[i]`, 因此导致排序错误。

```
liuchenxi@liuchenxideMacBook-Air q07 % python3 -m pdb merge_sort.py
> /Users/liuchenxi/Documents/系统开发工具/githubclone/lcx_course_work/w2/q07/merge_sort.py(1)<module>()
-> def merge(left, right):
(Pdb) l
1  -> def merge(left, right):
2      result = []
3      i = j = 0
4      while i < len(left) and j < len(right):
5          if left[i] <= right[j]:
6              result.append(left[i]); i += 1
7          else:
8              result.append(right[i]); j += 1 # 此处存在缺陷
9      return result + left[i:] + right[j:]
10
11 def merge_sort(a):
(Pdb) b 8
Breakpoint 1 at /Users/liuchenxi/Documents/系统开发工具/githubclone/lcx_course_work/w2/q07/merge_sort.py:8
(Pdb) c
> /Users/liuchenxi/Documents/系统开发工具/githubclone/lcx_course_work/w2/q07/merge_sort.py(8)merge()
-> result.append(right[i]); j += 1 # 此处存在缺陷
(Pdb) p i, j, left[i], right[j]
(0, 0, 3, 1)
(Pdb) n
> /Users/liuchenxi/Documents/系统开发工具/githubclone/lcx_course_work/w2/q07/merge_sort.py(4)merge()
-> while i < len(left) and j < len(right):
(Pdb) q
```

图 1.20: 使用 `pdb` 调试并观察变量

在图 1.20 中, `b 8` 表示在第 8 行设置断点, `c` 表示继续执行, `p` 表示打印表达式, `n` 表示单步执行下一条语句, `q` 表示退出调试器。通过观察 `i`、`j`、`left[i]`、`right[j]` 的值, 可以定位到缺陷: `else` 分支中应使用 `right[j]` 而非 `right[i]`。将 `merge` 函数中的 `result.append(right[i])` 修改为 `result.append(right[j])` 即可完成最小修复, 没有用 `sorted` 替换归并排序。

如图 1.21 所示, 编写 `test_merge_sort.py`, 增加两个测试: 题目给定的输入 `[3, 1, 4, 1, 5, 9, 2, 6]` 和含重复元素的列表 `[5, 2, 2, 7, 1, 1, 5]`。

```
liuchenxi@liuchenxideMacBook-Air q07 % vim test_merge_sort.py
liuchenxi@liuchenxideMacBook-Air q07 % cat test_merge_sort.py
from merge_sort import merge_sort

def test_given_input():
    """测试题目给定的输入"""
    assert merge_sort([3, 1, 4, 1, 5, 9, 2, 6]) == [1, 1, 2, 3, 4, 5, 6, 9]

def test_duplicate_elements():
    """测试含重复元素的列表"""
    assert merge_sort([5, 2, 2, 7, 1, 1, 5]) == [1, 1, 2, 2, 5, 5, 7]
```

图 1.21: 编写 `test_merge_sort.py` 测试

如图 1.22 所示, 运行 `pytest test_merge_sort.py -v`, 输出显示两个测试全部通过, 说明缺陷已被正确修复。

```

liuchenxi@liuchenxideMacBook-Air q07 % pytest test_merge_sort.py -v
===== test session starts =====
platform darwin -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- /opt/homebrew/Cellar/pytest/9.1.1/libexec/bin/python
cachedir: .pytest_cache
rootdir: /Users/liuchenxi/Documents/系统开发工具/githubclone/lcx_course_work/w2/q07
collected 2 items

test_merge_sort.py::test_given_input PASSED [ 50%]
test_merge_sort.py::test_duplicate_elements PASSED [100%]

===== 2 passed in 0.00s =====

```

图 1.22: 归并排序测试通过

1.3.2 实验结果

第七题的 github 的 commit 记录和 push 记录如图 1.23 所示。

```

liuchenxi@liuchenxideMacBook-Air w2 % git add q07
liuchenxi@liuchenxideMacBook-Air w2 % git commit -m "完成了 q07"
[main 2c74fcd] 完成了 q07
 4 files changed, 28 insertions(+)
 create mode 100644 w2/q07/__pycache__/merge_sort.cpython-314.pyc
 create mode 100644 w2/q07/__pycache__/test_merge_sort.cpython-314-pytest-9.1.1.pyc
 create mode 100644 w2/q07/merge_sort.py
 create mode 100644 w2/q07/test_merge_sort.py
liuchenxi@liuchenxideMacBook-Air w2 % git push
[Enter passphrase for key '/Users/liuchenxi/.ssh/id_ed25519':
Enumerating objects: 11, done.
Counting objects: 100% (11/11), done.
Delta compression using up to 10 threads
Compressing objects: 100% (9/9), done.
Writing objects: 100% (9/9), 2.78 KiB | 2.78 MiB/s, done.
Total 9 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:lcx821/lcx_course_work.git
 f1e18bb..2c74fcd main -> main

```

图 1.23: github 记录 Q7

1.4 第 8 题先测量，再优化慢速词频程序

1.4.1 实验内容

如图 1.24 所示，使用 vim 编辑器创建 generate_words.py 和 wordfreq.py。generate_words.py 使用固定的随机种子生成包含 1000 个不同词、共 30000 个词的 words.txt；wordfreq.py 通过列表 unique 逐词判断去重，统计不重复词的数量。

```

liuchenxi@liuchenxideMacBook-Air q08 % vim generate_words.py
liuchenxi@liuchenxideMacBook-Air q08 % cat generate_words.py
import random
random.seed(20260831)
vocab = [f"w{i}" for i in range(1000)]
with open("words.txt", "w", encoding="utf-8") as f:
    f.write(" ".join(random.choice(vocab) for _ in range(30000)))
liuchenxi@liuchenxideMacBook-Air q08 % vim wordfreq.py
liuchenxi@liuchenxideMacBook-Air q08 % cat wordfreq.py
words = open("words.txt", encoding="utf-8").read().split()
unique = []
for word in words:
    if word not in unique:
        unique.append(word)
print("count=", len(unique))

```

图 1.24: 创建 generate_words.py 和 wordfreq.py

如图 1.25 所示，运行 python3 generate_words.py 生成 words.txt。

```
liuchenxi@liuchenxideMacBook-Air q08 % python3 generate_words.py
```

图 1.25: 生成 words.txt

如图 1.26 所示, 使用 `time` 命令运行原程序 `wordfreq.py` 两次, 记录耗时。两次输出 `count=1000`, 耗时分别为 0.149s 和 0.127s。

```
liuchenxi@liuchenxideMacBook-Air q08 % time python3 wordfreq.py
count= 1000
python3 wordfreq.py  0.12s user 0.01s system 87% cpu 0.149 total
liuchenxi@liuchenxideMacBook-Air q08 % time python3 wordfreq.py
count= 1000
python3 wordfreq.py  0.11s user 0.01s system 97% cpu 0.127 total
```

图 1.26: 使用 `time` 运行原程序两次

如图 1.27 所示, 使用 `python3 -m cProfile -s cumulative wordfreq.py` 进行性能分析。从输出可以看到, 程序共 1012 次函数调用, 累计耗时 0.094s, 主要耗时集中在 `wordfreq.py` 第 1 行的模块级代码 (cumtime 0.094s), 即 `for` 循环中对列表 `unique` 的 `word not in unique` 判断。由于列表的成员判断复杂度为 $O(n)$, 整体复杂度为 $O(n^2)$, 这是程序的性能瓶颈。

```
liuchenxi@liuchenxideMacBook-Air q08 % python3 -m cProfile -s cumulative wordfreq.py
count= 1000
      1012 function calls in 0.094 seconds

Ordered by: cumulative time

ncalls  tottime  percall  cumtime  percall filename:lineno(function)
      1   0.000    0.000    0.094    0.094 {built-in method builtins.exec}
      1   0.093    0.093    0.094    0.094 wordfreq.py:1(<module>)
      1   0.000    0.000    0.000    0.000 {method 'split' of 'str' objects}
      1   0.000    0.000    0.000    0.000 {method 'read' of 'io.TextIOWrapper' objects}
    1000   0.000    0.000    0.000    0.000 {method 'append' of 'list' objects}
      1   0.000    0.000    0.000    0.000 {built-in method _io.open}
      1   0.000    0.000    0.000    0.000 <frozen codecs>:322(decode)
      1   0.000    0.000    0.000    0.000 {built-in method builtins.print}
      1   0.000    0.000    0.000    0.000 {built-in method _codecs.utf_8_decode}
      1   0.000    0.000    0.000    0.000 {method 'disable' of '_lsprof.Profiler' objects}
      1   0.000    0.000    0.000    0.000 <frozen codecs>:312(__init__)
      1   0.000    0.000    0.000    0.000 {built-in method builtins.len}
      1   0.000    0.000    0.000    0.000 <frozen codecs>:263(__init__)
```

图 1.27: 使用 `cProfile` 分析耗时

如图 1.28 所示, 创建优化后的 `wordfreq_opt.py`, 将去重用的列表 `unique` 改为集合 `set`: `unique = set(words)`。集合的成员判断平均复杂度为 $O(1)$, 可以把整体复杂度从 $O(n^2)$ 降到 $O(n)$, 且不改变最终的 `count` 结果, 也没有写死 1000。

```
liuchenxi@liuchenxideMacBook-Air q08 % vim wordfreq_opt.py
liuchenxi@liuchenxideMacBook-Air q08 % cat wordfreq_opt.py
words = open("words.txt", encoding="utf-8").read().split()
unique = set(words)
print("count=", len(unique))
```

图 1.28: 创建优化后的 `wordfreq_opt.py`

如图 1.29 所示, 使用相同输入再次运行优化后的程序两次, 耗时分别为 0.053s 和 0.043s。

```
[liuchenxi@liuchenxideMacBook-Air q08 % time python3 wordfreq_opt.py
count= 1000
python3 wordfreq_opt.py  0.02s user 0.01s system 55% cpu 0.053 total
[liuchenxi@liuchenxideMacBook-Air q08 % time python3 wordfreq_opt.py
count= 1000
python3 wordfreq_opt.py  0.02s user 0.01s system 74% cpu 0.043 total
```

图 1.29: 使用 `time` 运行优化程序两次

优化前两次耗时为 0.149s 和 0.127s，中位数为 0.138s；优化后两次耗时为 0.053s 和 0.043s，中位数为 0.048s。加速比为 $0.138/0.048 \approx 2.9$ ，即优化后性能约提升到原来的 2.9 倍，且最终 count 仍为 1000，结果保持不变。

1.4.2 实验结果

第八题的 github 的 commit 记录和 push 记录如图 1.30 所示。

```
[liuchenxi@liuchenxideMacBook-Air w2 % git add q08
[liuchenxi@liuchenxideMacBook-Air w2 % git commit -m "完成了q08"
[main c923246] 完成了q08
 4 files changed, 15 insertions(+)
 create mode 100644 w2/q08/generate_words.py
 create mode 100644 w2/q08/wordfreq.py
 create mode 100644 w2/q08/wordfreq_opt.py
 create mode 100644 w2/q08/words.txt
[liuchenxi@liuchenxideMacBook-Air w2 % git push
[Enter passphrase for key '/Users/liuchenxi/.ssh/id_ed25519':
Enumerating objects: 10, done.
Counting objects: 100% (10/10), done.
Delta compression using up to 10 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (8/8), 53.05 KiB | 2.53 MiB/s, done.
Total 8 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To github.com:lcx821/lcx_course_work.git
 2c74fcd..c923246  main -> main
```

图 1.30: github 记录 Q8

1.5 Github 仓库地址

本次课程的报告、练习、代码等，均已上传至个人 github 仓库。
Github 仓库地址：https://github.com/lcx821/lcx_course_work

课后练习内容和结果

本章记录课后实例练习的内容与结果，主要涵盖 Shell 的花括号展开、重定向、引号、退出码、作业控制、别名等基础概念，以及 Vim 编辑器、time 计时命令和 Python 调试器 pdb 的使用。

2.1 第 1 题花括号展开

2.1.1 实验内容

如图 2.1 所示，使用 `mkdir -p demo/{src,docs,tests}` 命令一次性创建三个目录，然后使用 `ls -F demo` 命令查看创建结果。

```
liuchenxi@liuchenxideMacBook-Air ms-practice % mkdir -p demo/{src,docs,tests}
liuchenxi@liuchenxideMacBook-Air ms-practice % ls -F demo
docs/  src/  tests/
liuchenxi@liuchenxideMacBook-Air ms-practice %
```

图 2.1: 花括号展开

在图 2.1 中，`{src,docs,tests}` 是花括号展开 (Brace Expansion)，Shell 在执行命令前会先把它展开成 `src`、`docs`、`tests` 三个字符串，因此整条命令等价于 `mkdir -p demo/src demo/docs demo/tests`。其中 `-p` 选项表示递归创建目录，父目录不存在时自动创建、已存在时也不报错。`ls` 的 `-F` 选项会在每个条目末尾追加类型标识符，其中 `/` 表示目录。输出 `docs/ src/ tests/` 说明三个目录均已创建成功。

2.2 第 2 题重定向

2.2.1 实验内容

如图 2.2 所示，先用 `echo "first line" > demo/out.txt` 将第一行内容写入文件，再用 `echo "second line" >> demo/out.txt` 追加第二行内容，最后用 `cat demo/out.txt` 查看文件内容。

```
$ echo "first line" > demo/out.txt
$ echo "second line" >> demo/out.txt
$ cat demo/out.txt
first line
second line
$
```

图 2.2: 输出重定向与追加重定向

在图 2.2 中, > 是输出重定向, 把命令的标准输出写入指定文件, 若文件已存在则先覆盖其内容; >> 是追加重定向, 把输出追加到文件末尾而不覆盖原有内容。因此第一次写入 first line、第二次追加 second line 之后, out.txt 中依次包含这两行内容。

2.3 第 3 题引号

2.3.1 实验内容

如图 2.3 所示, 先用 `foo=bar` 给变量 `foo` 赋值, 然后分别用双引号和单引号打印 `$foo`。

```
$ foo=bar
$ echo "$foo"
bar
$ echo '$foo'
$foo
$
```

图 2.3: 单引号与双引号的区别

在图 2.3 中, `echo "$foo"` 使用双引号, 其中的变量会发生展开, `$foo` 被替换成变量 `foo` 的值 `bar`, 所以输出 `bar`; 而 `echo '$foo'` 使用单引号, 单引号内的所有字符都按字面量原样处理、不进行变量展开, 所以原样输出 `$foo`。

2.4 第 4 题退出码

2.4.1 实验内容

如图 2.4 所示, 先用 `ls /no_such_dir_xyz` 列出一个不存在的目录 (命令失败), 再用 `echo $?` 查看上一条命令的退出码; 然后执行总是成功的 `true` 命令, 再次用 `echo $?` 查看退出码。

```
$ ls /no_such_dir_xyz
ls: /no_such_dir_xyz: No such file or directory
$ echo $?
1
$ true
$ echo $?
0
$
```

图 2.4: 查看命令的退出码

在图 2.4 中, `$?` 是 Shell 的内置变量, 保存上一条命令的退出码 (exit code), 其中 0 表示命令执行成功、非 0 表示失败。`ls` 访问不存在的目录时失败, 返回退出码 1; 而 `true` 是一条不做任何事、总是返回 0 的命令, 因此第二次 `echo $?` 输出 0。

2.5 第 5 题作业控制

2.5.1 实验内容

如图 2.5 所示, 依次演示了后台运行、暂停、恢复、终止等 Shell 作业控制操作。

```
$ sleep 1000 &
[1] 85565
$ jobs
[1] + running      sleep 1000
$ kill -STOP %1
[1] + suspended (signal) sleep 1000
$ jobs
[1] + suspended (signal) sleep 1000
$ bg %1
[1] + continued    sleep 1000
$ jobs
[1] + running      sleep 1000
$ kill %1
$
[1] + terminated   sleep 1000
$ jobs
$
```

图 2.5: 作业控制

在图 2.5 中, `sleep 1000 &` 末尾的 `&` 使命令在后台运行, 其中 [1] 是作业号、85565 是该进程的 PID。`jobs` 列出当前 Shell 的所有后台作业, 显示作业 1 正在运行 (running)。`kill -STOP %1` 向作业 1 发送 `SIGSTOP` 信号使其暂停 (suspended), 其中 `%1` 用作业号引用作业。`bg %1` 让暂停的作业在后台继续运行 (continued)。最后 `kill %1` 发送默认的 `SIGTERM` 信号终止作业 1 (terminated), 此时再执行 `jobs` 列表已为空。

2.6 第 6 题别名

2.6.1 实验内容

如图 2.6 所示，先用 `alias ll="ls -lh"` 定义别名，再用不带参数的 `alias` 查看已定义的别名，最后用 `ll demo` 使用该别名。

```
$ alias ll="ls -lh"
$ alias ll
ll='ls -lh'
$ ll demo
total 8
drwxr-xr-x  2 liuchenxi  staff   64B  9月  9 22:16 docs
-rw-r--r--  1 liuchenxi  staff   23B  9月  9 22:18 out.txt
drwxr-xr-x  2 liuchenxi  staff   64B  9月  9 22:16 src
drwxr-xr-x  2 liuchenxi  staff   64B  9月  9 22:16 tests
$
```

图 2.6: 定义并使用别名

在图 2.6 中，`alias 名称="命令"` 用于给命令起别名，之后输入别名就等价于输入原命令。`ls -lh` 中 `-l` 表示长格式（显示权限、大小、修改时间等信息），`-h` 表示以人类可读的单位（如 B、KB）显示文件大小。不带参数的 `alias` 会列出当前所有别名，可见 `ll='ls -lh'`。因此执行 `ll demo` 实际就是执行 `ls -lh demo`。

2.7 第 7 题 Vim 插入模式

2.7.1 实验内容

如图 2.7 所示，在 Vim 中进入插入模式输入 `print("Hello, Vim!")`；保存退出后，如图 2.8 所示用 `cat demo/hello.py` 查看文件内容。

```
print("Hello, Vim!")
~
~
~
```

图 2.7: Vim 插入模式

```
$ vim demo/hello.py
$ cat demo/hello.py
print("Hello, Vim!")
$
```

图 2.8: 保存并查看文件

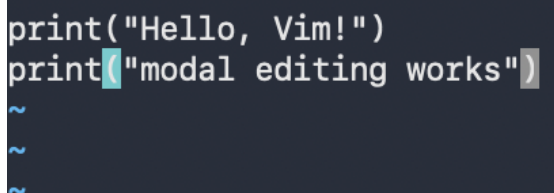
在图 2.7 和图 2.8 中，使用 `vim demo/hello.py` 打开（或新建）文件 `hello.py` 后默认处于普通模式，按 `i` 进入插入模式（insert mode），此时可以输入文本 `print("Hello, Vim!")`；

输入完成后按 `Esc` 返回普通模式，再输入 `:wq` 保存并退出。最后用 `cat` 命令查看文件内容，确认写入成功。

2.8 第 8 题 Vim 模态编辑

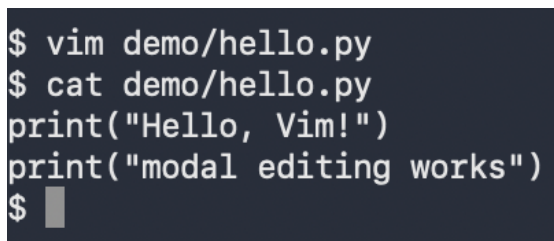
2.8.1 实验内容

如图 2.9 所示，在普通模式下按 `o` 在当前行下方新建一行并进入插入模式，输入第二行 `print("modal editing works")`；保存退出后如图 2.10 所示文件包含两行内容；最后如图 2.11 所示，用 `dw` 命令删除一个单词。



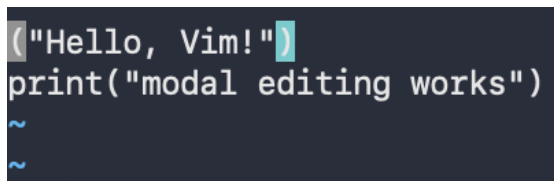
```
print("Hello, Vim!")
print("modal editing works")
~
~
~
```

图 2.9: 新建一行并输入内容



```
$ vim demo/hello.py
$ cat demo/hello.py
print("Hello, Vim!")
print("modal editing works")
$
```

图 2.10: 保存后的文件内容



```
("Hello, Vim!")
print("modal editing works")
~
~
```

图 2.11: 使用 `dw` 删除单词

在图 2.9 至图 2.11 中，Vim 是模态编辑器，普通模式下按键都是命令。`o`（小写 `o`）在当前行下方另起一行并进入插入模式，用于输入新的一行内容。`dw`（delete word）删除从光标位置到当前单词结尾的内容：把光标移到第一行 `print` 的 `p` 上按 `dw`，即删除 `print` 这一单词，使第一行变为 `("Hello, Vim!")`。

2.9 第 9 题 time 计时命令

2.9.1 实验内容

如图 2.12 所示，使用 `time` 命令测量 `sleep 1` 的执行时间。

```
$ bash -c 'time sleep 1'

real    0m1.008s
user    0m0.001s
sys     0m0.002s
$
```

图 2.12: *time* 命令计时

在图 2.12 中, *time* 命令用于测量命令的执行时间, 这里通过 `bash -c 'time sleep 1'` 在 `bash` 中执行。 `sleep 1` 会让进程暂停 1 秒。输出中的 `real` 是命令实际经过的总时间 (约 1 秒); `user` 是用户态 CPU 时间; `sys` 是内核态 CPU 时间。由于 `sleep` 大部分时间处于等待状态, `user` 和 `sys` 都接近 0。

2.10 第 10 题 *pdb* 调试器

2.10.1 实验内容

如图 2.13 所示, 先用 here-document 写入一个 Python 脚本 `loop.py`, 再用 `python3 -m pdb` 调试该脚本。

```
$ cat > demo/loop.py <<'EOF'
for i in range(3):
    print("i =", i)
    squared = i * i
print("done")
EOF
$ python3 -m pdb demo/loop.py
> /Users/liuchenxi/ms-practice/demo/loop.py(1)<module>()
-> for i in range(3):
(Pdb) l
  1 -> for i in range(3):
  2     print("i =", i)
  3     squared = i * i
  4     print("done")
[EOF]
(Pdb) n
> /Users/liuchenxi/ms-practice/demo/loop.py(2)<module>()
-> print("i =", i)
(Pdb) n
i = 0
> /Users/liuchenxi/ms-practice/demo/loop.py(3)<module>()
-> squared = i * i
(Pdb) p i
0
(Pdb) n
> /Users/liuchenxi/ms-practice/demo/loop.py(1)<module>()
-> for i in range(3):
(Pdb) n
> /Users/liuchenxi/ms-practice/demo/loop.py(2)<module>()
-> print("i =", i)
(Pdb) p i
1
(Pdb) q
$
```

图 2.13: 使用 *pdb* 调试 Python 脚本

在图 2.13 中, `cat > demo/loop.py <<'EOF' ... EOF` 是 here-document (此处文档), 把 EOF 之间的内容原样写入 `loop.py` 文件, 其中 `<<'EOF'` 的单引号表示不对内容做变量展开。写入的脚本为 `for i in range(3)` 的循环, 循环体依次打印 `i` 的值并计算 `squared = i * i`, 循环结束后打印 `done`。

`python3 -m pdb demo/loop.py` 使用 Python 内置的调试器 `pdb` 运行脚本, 进入交互式调试。其中各调试命令的含义如下: `l` (`list`) 列出当前所在位置的源码; `n` (`next`) 单步执行当前行、不进入函数内部; `p i` (`print`) 打印变量 `i` 的值, 图中第一次输出 `0`、第二次输出 `1`; `q` (`quit`) 退出调试器。

解题感悟

这周在经过上周的练习后，对命令行操作这些流程更加熟悉了。解决问题也没有上次那么痛苦了。虽然还是有一些问题需要查阅资料，但至少比上一次清楚了很多。

然后因为上次已经配置好了 `latex` 环境，学习了怎么使用 `latex` 写作，这次实验报告的撰写也明显顺利了许多，完全没有上次那么痛苦了。

希望经过这几次的练习，下次完成练习能更加顺利，以后也能更加熟练的使用命令行操作和 `latex` 写作。

